Project Brief: Open-Source Multi-Vehicle Diagnostic, Triage, & Calibration Tool

1. Project Mission & Objective

The goal is to build a **completely open-source, multi-layered diagnostic, triage, and calibration tool** capable of servicing both legacy Internal Combustion Engine (ICE) vehicles and modern Electric Vehicle Supply Equipment (EVSE) environments. Commercial calibration software and EV diagnostic platforms cost thousands of dollars in licensing. This project democratizes vehicle and infrastructure service by building an accessible Python-based suite. The application will isolate vehicle communication failures, handle raw ECU binary file read/write map modifications, validate file integrity via checksums, and diagnose EV handshake/financial transaction bugs across the entire charging ecosystem:

By leveraging Cursor's built-in execution environment for mock validation, this tool allows developers and technicians to test software logic without requiring physical test benches or expensive licensing.

2. The Four Pillars of the Tool's Architecture

Pillar A: ICE ECU Tuning, Hex Editing & File Validation

- **The Problem:** Modifying engine parameters (deleting Diagnostic Trouble Codes, modifying ignition/fuel maps, adjusting limiters) requires expensive binary editing software. Furthermore, modifying a raw `.bin` file without recalculating its cryptographic **checksum** can brick the vehicle's ECU upon writeback.
- **The Open-Source Solution:**
    - **File I/O Stream (`io.FileIO`):** Python scripts to securely open, read, and write raw binary EEPROM/Flash dumps (`.bin`, `.hex`, `.s19`).
    - **Hex/Map Hunting Matrix:** Custom Python parsing algorithms to scan hex files for known structured patterns (e.g., 2D/3D maps, axes, and DTC tables).
    - **Checksum Validation:** Implementation of open-source algorithmic checksum modules (such as standard CRC16, CRC32, or Bosch-specific sum checks) to verify file integrity *before* flagging a file as safe to flash.

Pillar B: Vehicle Telematics & Live OBD-II Stream

- **The Problem:** Isolating whether a fault lies within a vehicle's internal modules or external communications requires real-time parametric visibility.
- **The Open-Source Solution:**
    - `python-OBD` **&** `python-can`**:** Interfacing with low-cost ELM327 or STN-based pass-thru adapters to pull real-time Parameter IDs (PIDs) and read/clear Diagnostic Trouble Codes (DTCs).

Pillar C: EVSE Station-to-Network Communications (OCPP)

- **The Problem:** Charging drops out mid-session, authorization code timeout, or the station undergoes "silent freezes" due to communication protocol mismatches.
- **The Open-Source Solution:**
    - ocpp **(Python library):** Full implementation of Open Charge Point Protocol (OCPP 1.6J and 2.0.1 via WebSockets) to act as a local mock Central System (CSMS) proxy, exposing broken raw JSON messages.

Pillar D: Financial & Telematics Billing Logic

- **The Problem:** The physical hardware works, but transaction handshakes fail due to bad payment gateway syncing or invalid authorization payload syntax.
- **The Open-Source Solution:**
    - Capturing and analyzing backend JSON authorization requests (Authorize.req, StartTransaction.req) to pinpoint exactly where payment processing or digital certificates fail.

3. Recommended Open-Source Stack & Internal Emulation

To keep this project entirely free and lightweight, the environment utilizes Cursor's built-in ecosystem rather than external microcontroller emulators:

Layer	Component	Free / Open-Source Tool
Development IDE	Code Engine & Sandbox	**Cursor** (Using integrated terminal & AI context)

Emulation & Testing	Internal Validation System	**Python Mock Libraries** (unittest.mock) executed directly inside Cursor's terminal environment to simulate live ECU data streams and WebSocket payloads.
ICE Binary Layer	Hex/Map/Checksum Processing	Custom Python scripts using numpy (for multi-dimensional array mapping) and standard hex parsing libraries.
Vehicle Interface	OBD-II Data Pipeline	python-obd and python-can
Station Interface	Network Protocol Layer	ocpp via asynchronous WebSockets (websockets)

Frontend GUI	Light, User-Friendly UI	**CustomTkinter** (for a modern, dark-mode, lightweight desktop layout)

4. Minimum Viable Product (MVP) Dashboard Layout

The GUI will feature a clean tabbed panel designed for rapid field triage:

1. **Tab 1: Binary Studio (ICE Modification)** * A file upload interface to load a `.bin` file.
    - A visual Hex viewer showing address offsets.
    - A **"Validate Checksum"** button that outputs a pass/fail indicator based on the file type.
2. **Tab 2: Telematics Live Stream (OBD-II)**
    - Real-time text output stream for reading PIDs, tracking battery State of Charge (SoC), and viewing active engine DTCs.
3. **Tab 3: OCPP Network Sniffer (EV Environment)**
    - A live-logging window that captures incoming and outgoing WebSocket communication frames, color-coded for fast triage (Green for Accepted requests, Red for Failed authorizations/Drops).

5. How You Can Help Me Build This

Message to the Developer:

I possess the core diagnostic and automotive system experience to know exactly what parameters, maps, and network protocols break down in the field. I need your assistance structuring the Python application architecture.

We will bootstrap this project directly within **Cursor**, using its built-in terminal and mock scripts to validate our code logic safely without needing physical hardware. We will start by creating modular scripts for the **Binary Hex Editor / Checksum validator** and the **OCPP network listener**, then wrap them together in a clean **CustomTkinter** user interface.